

Chapter 2 Lecture 1 - Lecture 10 (buX)

- Why do we multiply the index of an array by 4 to get the offset in order to retrieve data from memory to register or to store data from register to memory?

Ans: Array is stored in main memory. In main memory each slot contains 8 bit data or 1 byte data. As MIPS is 32 bit architecture data's will also be of 32 bit. So to retrieve 32 bit or 1 word data from main memory we will need 4 data slots. So address will be multiple of 4. For example, if $A[0]$ is in 0 slot then $A[1]$ will be in 4 and so on.

- Can we subtract an integer from a value stored in a register? Give an example.

Ans: we can not directly subtract an integer from a value stored in a register. By directly I meant we can not use SUB command. We can do it with ADDI. though ADDI is used to add an integer value with a value stored in register. But we can take a negative integer and do the subtraction. For example: suppose 3 is stored in $\$s1$, now let's write a code for $3-1$, it will be: `addi $s1, -1`

- Write the MIPS code to move a value from $\$s3$ to $\$t0$.

Ans: `add $t0, $s3, $zero` { $\$zero$ is a register which contains 32 zeroes. It is used for moving between data}

- In the case of signed numbers, why can't we represent $-(-2^{(n-1)})$

Ans: it can be written as $2^n - 1$. In signed number MSB bit is 0 for positive number. For simplicity let's assume n is 8 (8 bit). So $2^n - 1 = 128$, Binary of 128 is 01000 0000. Here 0 is for positive sign. Now we can see that the binary is of 9 bit. But it was supposed to be 8 bit. Because of sign bit in binary we can not represent $-(-2^{(n-1)})$. This is same for any number of bits (n).

- Consider the following MIPS code: `srl $t7, $t0, 10`; what type of instruction is this? Write the machine code for this instruction.

Ans: this is an R-format instruction. [rs stays 0 in shift operation. `srl rd, rt, shamt`]

op	rs	rt	rd	shamt	funct
000000	00000	01001	10001	01010	000010

MIPS instructions are encoded in binary, as 32-bit instruction words, called machine code. The layout of an instruction is called the instruction format. Only 3 different formats exist.

- In the I-type instruction, the rt 5 bits can hold both source and destination register numbers. For which instruction does it hold the source register number? For which instruction does it hold the destination register number? Give an example of each.

Ans: I-format is used for constants or whole numbers. Example: `rt=source`, instruction: `sw $t1, 24($s1)` and `rt=destination`, instruction: `lw $t0, 12($s0)`

You can ignore this part. Only for better understanding: for different code the format is:

`addi destination(rt), constant(rs)`

`Lw destination(rt), offset (source(rs))` || `sw source(rt), offset (source(rs))` (for `lw` and `sw` base address will always be in `rs`)

- What does the Instruction Address Register hold? Why do we add 4 to it when calculating the branch destination address?

Ans: Program counter is called as instruction address register. It holds instruction pointer. It increases $PC+4$. As every instruction is 4 slots apart in main memory, PC adds 4 with itself to hold the address of next instruction.

- Suppose the data 1234ABCD is stored in the memory in the following way-

0x4AB00100	CD
0x4AB00101	AB
0x4AB00102	34
0x4AB00103	12

- Is this a Big-Endian system or Little-Endian system? Why?
- What do you understand by "The Memory is Word Aligned in MIPS"?

Ans: i) this is little Endian system. Because LSB of the data is stored in the least address.

ii) the address of every word must be the multiple of 4. This is known as word alignment. And maintaining this rule is known as alignment restriction. Each slot of mips main memory is of 8 bit. so we need 4 slots for one word. That is why main memory of mips is word aligned.

9. Suppose the data in each array index is 16 bits. To push this data into the stack, how much should we increment or decrement the stack pointer? Why?

Ans: [not sure] as we know for 32 bit data we needed 4 memory slot where each slot is of 8 bit. So for 16 bit data we will need 2 memory slot. That is why to insert or push a data stack pointer should decrement by 2.

10. What is the smallest value we can subtract using an MIPS immediate instruction?

Ans: mips is 32 bit architecture. to subtract an immediate instruction we will need a negative value. so the smallest negative value for subtraction is $-2^{(n-1)} = -2^{32-1} = -2147483648$

11. Suppose you have a switchboard that has 8 switches for turning the lights on and off. The current state of the switches can be represented by the binary number: 10110011 where 1 means the corresponding switch is on and 0 means it is off. Now using MIPS instructions, you wish to automate the process of turning on and off. How can you turn off the 3rd and 4th switch from the left and keep everything else as is using MIPS instructions?

Ans: suppose, switch state is in \$t1, now to make 3rd and 4th from left to be turn off, we will have to do And operation. Mips instruction will be: and \$t0, \$t1, \$t2, where \$t2= 10000011 then the result will be \$t0=10000011

\$t2	1000 0011
\$t1	1011 0011
\$t0	1000 0011

12. What type of instruction is SLTI? How can you implement $\geq$ using MIPS instructions? Give an example code.

Ans: this is an I-type instruction. $\geq$ can be done with slti and beq. Example: here, \$t1=4

Beq \$t1,5,exit1 [checking if 4==5, if its true code will jump to exit. if false then it will go to next line]

Slti \$t0,\$t1,5 [here, if t1(4)<5 then t0=1]

Bne \$t0,\$zero,exit [for 4>5 t0 should be 0. So if not then t0 will be 1 and the code will jump to exit]

exit1:

Code [other lines of code]

exit:

13. Suppose the newly created BRACS Architecture System supports word size of 64 bit and each memory cell in the architecture is of size 8 bit. In this architecture, what will be the location of an array element Arr[6] if the base address of the array is 0x11224814?

Ans: $8/64 = 8$ slot. So we will need 8 slot in the memory for 64bit data. Then the location will be: offset=6*8=48
Then the location should be offset+base address. 48(0x11224814)

14. Let's assume instruction J 1020 is in memory address 0x00223380. Now calculate the Jump target address and write it down in hexadecimal

Ans: $1020 * 4 = \text{FF0}$

15. How can we branch to a far address using traditional branching instructions?

Ans: address is of 16 bit. but sometime we need to branch somewhere what can not be reached with 16 bit. In that case we mix match the traditional branching with jump instruction. as jump has address of 26 bits. that is why we can branch far than normal.

16. Give an example of register addressing. Ans: Add \$9,\$10,\$11

17. Give an example of immediate addressing. Ans: addi \$9,\$10,4

18. Give an example of Base addressing. Ans: lw \$t0,24(\$s0)

19. Give an example of PC-relative addressing. Ans: beq \$t0,\$t1,label

20. Give an example of Pseudodirect addressing. Ans: j label

21. In case of 20-bit architecture what is the data length for instruction lw,lb,lh?

ans: lw=20 bits, lb=8bit lh=10bit

22. In case of 30-bit architecture what is the data length for instruction lw,lb,lh?

ans: lw=30 bits, lb=8bit lh=15bit

23. In the case of 120-bit architecture what is the data length for instruction lw, lb, lh?

ans: lw=120 bits, lb=8bit lh=60bit

24. What is the difference between instruction lh and lhu

Ans: lh is for signed and lhu is for unsigned numbers.

25. What is the difference between conditional and unconditional branching? Give examples.

Ans: in conditional branching if a condition is true then the program jumps to a specific part. But for unconditional branching, no condition is needed. It will jump to the specific part without any condition. Moreover, Conditional Branching is I-type instruction. And Unconditional branching is J-type instruction

26. Why in case of calculating branch or jump target address we perform a 2-bit left shift on the offset value?

Ans: to calculate address in the memory we used to multiply 4 with offset. So multiplying by 4 means 2 bit left shift. as $2^2=4$. That is why we 2bit left shift on the offset value.

27. In a 64-bit architecture what will be the length of the registers. Draw the diagram of the register file (having all the input and output links with valid bit lengths) having 128 128-bits registers.

Ans:

28. What type of data is stored in static data part of the memory?

Ans: data known at compile time. Global or static variables, Constants and strings.

29. What type of data is stored in text part of the memory?

Ans: program code or user level code.

30. What type of data is stored in dynamic data part of the memory

Ans: data allocated during runtime.

31. Which instruction should we use to retrieve a 1byte integer value from B[6]?

Ans: lbu rt, offset(rs)

32. Suppose, we want to check if the value of \$t1 is greater than 66. Which instruction can we use and how does it check?

Ans: slti rd, rs, constant; so the instruction will be:

slti \$t0, \$t1, 66 //if t1<66 then t0=1 else 0

Bne \$t0, \$zero, exit //if t1>66 then t0=0

Code //if t1>66 then it won't go to exit directly.

Exit:

33. Which instructions can we use to compare (Equality, Less than, Greater than) the values of two registers and how do they work?

- For less than we can use slt and beq. From slt if its true then we will get 1, so with beq we can check if it returns 0 then we can jump to exit. means condition false. if its true it will go to next line.
- For greater than we can use slt and bne. From slt if its true then we will get 1, so with beq we can check if it returns 0 then we can jump to exit. means condition false. if its true it will go to next line.
- For equality we can use bne. [if it is not equal it will jump to exit. and if equal then the code will run from next line]

34. What does the \$sp hold? How can we retrieve the second value from the stack (If we consider the first value as stack top) ?

Ans: sp register holds the address of the top of the stack. as data in stack are word aligned. Each data needs 4 slot in the memory. so to push or pop we need to subtract or add 4. So to retrieve the second value from the stack we need to: addi \$sp, \$sp, 4 [means adding 4 with stack pointer or \$sp]. so now the SP will point towards the 2nd value. Now we can load it with lw command [lw \$s0, 0(\$sp)]

35. If we want to store a 16 bit value in stack, what should be the change in \$sp? What should be the instruction to store the value?

Ans for 16 bit we will need 2 slot of 8bits. So to push a value we need to subtract 2 from \$sp and then store it with sw. The instruction should be:

Addi \$sp, \$sp, -2

Sw \$s0, 0(\$sp)

36. What is a non-Leaf procedure? Give an example.

Ans: when a procedure calls other procedures than it's a non-leaf procedure. It restores from the stack after the call. It's the same as nested calls. Example:

C code:

```
int fact (int n)
```

```
{
```

```
    if (n < 1) return f;
```

```
    else return n * fact(n - 1);
```

```
}
```

Argument n in \$a0

Result in \$v0

■ MIPS code:

fact:

```
    addi $sp, $sp, -8      # adjust stack for 2 items
    sw   $ra, 4($sp)       # save return address
    sw   $a0, 0($sp)       # save argument
    slti $t0, $a0, 1       # test for n < 1
    beq  $t0, $zero, L1    # if so, result is 1
    addi $v0, $zero, 1     # pop 2 items from stack
    addi $sp, $sp, 8       # and return
    jr   $ra
L1: addi $a0, $a0, -1       # else decrement n
    jal  fact              # recursive call
    lw   $a0, 0($sp)       # restore original n
    lw   $ra, 4($sp)       # and return address
    addi $sp, $sp, 8       # pop 2 items from stack
    mul  $v0, $a0, $v0     # multiply to get result
    jr   $ra              # and return
```

Chapter 3

1. Which of the following are normalized numbers:

a. 0.000112344 = denormalized

[number is normalized only when it has only one non zero before the decimal point]

b. 11.1033 * 10³ = denormalized

c. 1.0011 * 2⁻⁵ = normalized

2. Suppose the following code snippet is the machine code of an ADA program:

0x13456688	SUB \$t0, \$t1, \$s1
0x1345668C	ADDI \$t0, \$t3, 1354545
0x13456690	SW \$t0, 0(\$t5)
0x13456694	JR \$t1

a. If PC = 0x1345668C and this is the instruction causing an overflow, what should be the value of EPC?

b. Why was ADDI instruction used instead of ADDIU?

Ans: a) EPC = exception program counter register. When a program faces overflow, PC is being saved in EPC. mfc0 can retrieve EPC value to return after handling the exception.

b) Addi generates an exception when it encounters overflow. Addiu does not generate any exception during overflow.

As we know ADA language requires raising an exception, we cannot use ADDIU for it.

3. Why do we shift the multiplicand register to the left in the long multiplication approach?

Ans: we shift multiplicand 1 left bit because during normal multiplication we use X after each digit. This x is shifting left by one bit in every iteration or digit. For clear understanding watch week 4 bangla supplementary videos [Long Multiplication part] 23 mins 32 sec.

4. If the exponent field of a 40-bit floating point number is 15 bit then what should be the bias for the exponent?

Ans: $\text{bias} = (2^{(n-1)}) - 1$, where n is bit number of exponents. So for 15bit, $\text{Bias} = 2^{(15-1)} - 1 = 16383$

5. What is overflow range if the exponent field is 7-bit

Ans: 7bit unsigned binary range = 0 to $(2^7) - 1 = 0$ to 127, here 0000000 and 1111111 will be reserved so the range should be 1 to 126. So the range of exponent (power of 2) should be between -62 to 63 for overflow range. (bias of 7 bit is 63)

6. What is overflow range if the exponent field is 11-bit

Ans: 11 bit unsigned binary range = 0 to $(2^7) - 1 = 0$ to 2047, here 00000000000 and 11111111111 will be reserved so the range should be 1 to 2046. So the range of exponent (power of 2) should be between -1022 to 1023 for overflow range. (bias of 11 bit is 1023)

7. What is overflow range if the exponent field is 8-bit

8. Ans: 8bit unsigned binary range = 0 to $(2^8) - 1 = 0$ to 255, here 00000000 and 11111111 will be reserved so the range should be 1 to 254. So the range of exponent (power of 2) should be between -126 to 127 for overflow range. (bias of 8 bit is 127)

9. What is overflow range if the exponent field is 9-bit

Ans: 9bit unsigned binary range = 0 to $(2^8) - 1 = 0$ to 511, here 000000000 and 111111111 will be reserved so the range should be 1 to 510. So the range of exponent (power of 2) should be between -254 to 255 for overflow range. (bias of 9 bit is 255)

10. What determines the number of iterations in both the long multiplication approach and optimized multiplication?

Ans: number of bits in multiplier is equal to iteration in both long and optimized multiplication.

11. in the 32 bit MIPS architecture, what does it mean by 64 bit product register?

ans: if we multiply two 32 bit numbers then their product will be 64bits. length of the product must be the sum of operand lengths. but mips is 32bit architecture, so there is no 64bit direct register. Here the 64 bit product register is made with 2 32 bit register. after multiplication the 32 MSB bit is stored in hi register and 32bit lsb is stored in lo register.

12. Which register value does not change in optimized multiplication?

Ans: multiplicand value stays same.

13. State the major differences between the long multiplication and optimized multiplication procedure.

ANS: in optimized multiplication it performs add and shift steps in parallel. but it's not parallel in long method. optimized multiplication has no use for multiplier register. it only takes product and multiplicand. In long method product is 64 bit but in optimized it's only 32 bits because in MSB only 32 bits are being added up.

14. Which instruction can you use to store the MSB 32 bit of product in \$rd?

Ans: `mult rs, rt` // `Rs*rt`, stores 64 bit product in hi and lo, For unsigned `multu rs, rt` can be used
`mfhi rd` // move from hi to rd. don't forget to put \$ before register number. here it is in general term.

15. Why is 'mul' a pseudoinstruction? Explain.

Ans: the `mul` pseudo instruction makes it look as if MIPS has a 32-bit multiply instruction that places its 32-bit result. it stores the lsb to rd. the bits of hi are not examined nor saved. so we can not check overflow. Because we can test hi value to see if product overflows 32bits. `mul rd, rs, rt` is same as doing
`mult rs, rt` then `mflo rd`. that is why this is a pseudoinstruction.

16. To avoid overflow, what type of instructions should be used?

Ans: `addu`, `addui`, `subu` can be used to avoid overflow in mips. As this all are unsigned.

17. What is the use of `cvt.w.s` instruction? When is it used?

Ans: The `cvt.s.w` and `cvt.w.s` instructions are used to convert between integer and floating point representations. The instruction: `cvt.w.s $f4, $f2` converts the contents of register \$f2 from single precision floating point representation to an integer in \$f4.

The instruction: `cvt.s.w $f4, $f2` converts the contents of register `$f2` from integer representation to single precision floating point number in `$f4`.

18. In the 32 bit MIPS architecture, how does the coprocessor registers allocate double precision float?

Ans: we know in MIPS, coprocessor has 32 bit register for both floating point and integer values. So for 64 bits or double precision it uses 2 consecutive registers.

19. What is the difference between storing an integer and storing a floating-point value in the main memory?

Ans: This is a broad discussion about the whole topic: Register file has 32 registers for integer values. It can only hold integers. Then we have main memory. Where dynamic data's are stored. Main memory can have both integer and floating point values. When main memory wants to do some arithmetic operation things it loads in register file. For simplicity when storing something from register to main memory we use `sw` operation. And for loading something from main memory to register file we use `lw` operation. So for floating point we have coprocessor 1. It has 32 bit register for floating point values. Coprocessor can hold integer or floating point both. Register file registers are `$s0...$s7` (saving variables) and `$t0...$t9` (temporary variables)

Coprocessor has `$f0` to `$f31` name registers.

To store coprocessor 1 to main memory we use `swc1`.

To load from main memory to coprocessor 1 we use `lwc1`

Coprocessor and register file can transfer between them self. To transfer a value from coprocessor to register file we must convert the floating point value into an integer with `mfc1 $s0, $f2`. But here `$f2` must contain an integer value. We will need `cvt.w.s $f4, $f2` to convert `f2` into an integer. This conversion happens in coprocessor 1. But to transfer an integer from register to coprocessor 1 we can directly transfer it as coprocessor 1 can also hold integer. The `mtc1` can move register file's value to coprocessor's register. For example: `mtc1 $s0, $f2` (First operand is integer register, next one is floating point register).

Chapter 4

1. In a single cycle Datapath which instruction sets the duration of the clock cycle and why?

Ans: `load` or `lw` takes the maximum time during an operation. So it will set the duration of the cycle. Or if `lw` is not there then the instruction that will take the longest time will set the duration of the clock cycle.

2. During the fetch cycle which memory is accessed and what operation is performed in that memory?

Ans: During fetch cycle PC provides us the memory address inside the instruction memory. From where instruction will be fetched. The hardware needed for instruction fetch are PC, Instruction memory and adder for incrementing PC with 4.

3. What is the length of control bits feeded into the control unit during R-type instruction and during I-type and J-type instructions?

R-type = I-type = J-type = op code = 6 bit [31 to 26]

4. After which stage PC value gets incremented and by what amount?

Ans: When PC sends address to instruction memory it also sends the address to an adder to get incremented by 4.

5. What control signals go to the register file and data memory during the execution of R-type, `lw` and `sw` instructions?

Ans: From control unit to register file it has `regwrite` control signal and from Control unit to data memory it has `memread` and `memwrite`.

R-type: `regwrite` = 1 `Memread` and `memwrite` = 0

Lw: `regwrite` = 1 and `memread` = 1 and `memwrite` = 0

Sw: `regwrite` = 0 `memwrite` = 1 and `memread` = 0

6. For a 64-bit Architecture, if each slot of instruction memory can hold 1 byte, what should be the increment value for the PC to get the next instruction?

Ans: 8 (As $64/1\text{byte}(8\text{bit})=8$)

7. For the beq instruction, what should be the value for ALU zero wire if the PC gets updated to branch destination address?

Ans: if two value is equal in beq then zero wire generates 1 otherwise 0. here pc is getting updated with 4 so zero wire must give 1 and branch will give 1. So output from and gate will be 1. this 1 will go through PCSrc wire and it will select pc updated to branch destination address.

8. Suppose, for an instruction set, add takes 5 ns, lw takes 6 ns and sw takes 10 ns to execute. What should be the clock period for this instruction set?

Ans: 10ns. Max time should be the clock period.

9. What is the use of ALU in lw instruction?

Ans: to load data from memory we add offset with base address. so from the lw machine code 16 bit address will go through a sign extend hardware and get converted to 32 bits. then alu will send 1 with alusrc wire. so the 32 bit sign extended offset will go to ALU. And value of base address will go through read reg 1 of register file to read data 1 to ALU. Now Alu will add base address and offset together and send the result to address of Data memory segment. then from data memory that data will get out from read data wire and go to write data of register file with the help of memtoreg=1 signal of control unit.

10. Depending on the instructions (R type/ lw, sw, addi/ beq), what does the ALU calculate?

Ans: Rtype: read data 1 and read data 2 will go to alu and do arithmetic operations.

lw is already discussed in ques 9

Sw=same as lw

Addi= adds value of source and constant by getting value from register file read data 1 and constant will get sign extend to 32 bit and then it will go to alu through a mux. This mux will select 2nd value of alu. The result from alu is send to write data of register file.

Beq= alu decides If pc+4 will go to pc or branch address will go. it calculates if 2 value is equal or not. alu will minus two value and if the result is 0 then two value are same. alu zero flag is 1 when result is 0. then this value is send to an and gate where this value meets alusrc value. Then result of and is pcsrc which goes to to decide if it will go to branch address or pc+4.